CZECH TECHNICAL UNIVERSITY IN PRAGUE
Faculty of Electrical Engineering
Department of Control Engineering

FOC Firmware for Integrated BLDC Driver

Bachelor's thesis

Mathias Palme

Supervisor
Ing. Krištof Pučejdl

2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Krištof Pučejdl katedra řídicí techniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **26.01.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Palme Mathias

Podpis studenta

© ČVUT v Praze, Design: ČVUT v Praze, VIC

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Pokyny pro vypracování:

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver with an STM-32 type microcontroller.
- Familiarize yourself with principles of FOC algorithm and review the existing implementations that use similar electronic components.
- Familiarize yourself with the specific uFOC hardware components their function, and the overall board layout and schematic.
- Develop and implement a firmware allowing the uFOC driver to control a BLDC motor using the FOC principle. Implement control modes for commanding output Torque, Velocity, and Position, and communication protocol via CAN bus, allowing setting, commanding and reading driver status from different devices connected on the bus.
- Create a documentation for the firmware on the existing GitHub repository.
- Test the firmware and demonstrate the control via simple benchmark application (for example a simple haptic device).

Seznam doporučené literatury:

[1] R. Gabriel, W. Leonhard and C. J. Nordby, „Field-Oriented Control of a Standard AC Motor Using Microprocessors," in IEEE Transactions on Industry Applications, 1980, doi: 10.1109/TIA.1980.4503770.
[2] A. Skuric et al., „SimpleFOC: A Field Oriented Control (FOC) Library for Controlling Brushless Direct Current (BLDC) and Stepper Motors," in Journal of Open Source Software, 2022, doi: 10.21105/joss.04232.
[3] R. C. Prayogo, A. Triwiyatno and M. A. Riyadi, „Field Oriented Control Implementation on BLDC Motor Controller with PI and SVPWM using STM32F103C8T6," in Journal of Physics: Conference Series (Vol. 2622). Institute of Physics, 2023, doi:10.1088/1742-6596/2622/1/012025

 © ČVUT v Praze, Design: ČVUT v Praze, VIC

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Mathias Palme
May 2026

Acknowledgments

Acknowledgment section need not to be too strict. Try to express your personality, sense of humor and so on :).

Abstract

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller developed by Bc. Šimon Pecháček and Ing. Krištof Pučejdl at the Department of Control Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague. The firmware implements FOC control algorithm to control torque. On top of torque control a PI velocity and PID position control loop is implemented. Aditionally communication via CAN bus is implemented with daisy chaining of multiple drivers, to allow indepemendent control and sensor readings from each driver via a signle CAN bus line.

Keywords: FOC, BLDC, embedded software, motor control.

Abstrakt

Česká verze abstraktu (anotace) s klíčovými slovy na konci. Doplnit českou verzi...

Klíčová slova: antikolizní systém, digitální mapa, kalmanův filtr, V2X-komunikace.

Contents

1 Introduction **1**

2 Introduction to FOC - work name **2**

2.1 FOC overview . . . 2
2.2 Clarke transformation . . . 3
2.3 Park transformation . . . 3
2.4 PI regulator . . . 4
2.5 Space vector modulation . . . 4
2.6 Velocity and position control loops . . . 4
2.7 Electrical angle calculation . . . 4

3 Firmware implementation **5**

3.1 Used technologies and development tools . . . 5
3.2 Firmware topology . . . 5
3.3 MCU configuration . . . 6
3.3.1 System clock . . . 6
3.3.2 Timer configuration . . . 6
3.3.3 ADC configuration . . . 7
3.3.4 PWM-ADC synchronization . . . 7
3.3.5 Communication interfaces . . . 7
3.3.6 SPI interfaces . . . 7
3.4 Driver . . . 8
3.4.1 Current sensing offset calibration . . . 8
3.5 Encoder . . . 8
3.5.1 Data reading . . . 8
3.5.2 Direction inversion . . . 8
3.5.3 CRC control . . . 8
3.5.4 Velocity calculation . . . 8
3.5.5 Signal filtering . . . 8
3.5.6 Electrical offset calibration . . . 8
3.5.7 Implemented functionality . . . 8
3.5.8 Electrical offset calibration . . . 8
3.6 FOC implementation . . . 8
3.6.1 Current control loop . . . 9
3.6.2 Velocity and position control loops . . . 9

3.7 Configuration abstraction layer 9

4 Conclusion **10**

4.1 Future work 10

Appendices **11**

A Supplementary figures **12**

B Advance things **13**

References **15**

1 | Introduction

BLDC motors are increasingly more popular in varous applications, such as robotics, drones, electric vehicles and many more, due to their high efficiency, ability to produce large torque at low speeds and high position accuracy. Unlike brushed DC motors, BLDC motors need specialized driver and control to achieve commutation.

While commercial BLDC drivers are available on todays market, such as ODrive Micro, moteus-c1 (add reference), they are often expensive in range of 70-90 USD per driver. Therefore, the Department of Control Engineering at the Czech Technical University in Prague has developed an integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller, which is a low-cost alternative to commercial drivers at BOM price of approximately 20 USD.

To achive an efficient commutation and generate torque control, the driver needs to implement a closed loop control. The most common control method for BLDC motors is Field Oriented Control (FOC), which allows for precise control of the motor's torque and speed by controlling the current in the stator windings.

Goal of this thesis is to develop embedded firmware that implements FOC current control, speed and position control loops and communication via CAN bus for the uFOC driver. Additionally a Python client side example for CAN communication with the driver is provided.

2 | Introduction to FOC - work name

2.1 FOC overview

Field Oriented Control is a closed loop control method for controlling torque of BLDC motors based on Clarke-Park Transformation. Standard BLDC motor is aquipped with three stator windings, which are supplied with three phase currents. To achive commutation and generate torque, propper voltage has to be provided to the stator windings by reading the phase currents (i_a, i_b, i_c). Three phase currents reference frames can be decomposed into two orthogonal torque and flux reference frames using Clarke transformation, resulting in two currents (i_α, i_β) in a stationary reference frame.

Park transformation is then applied to transform the currents from a stationary reference frame to a rotating reference frame, resulting in two DC currents (i_d, i_q), where i_d is a flux current and i_q is a torque current.

To control i_d and i_q currents, two independent proportional-integral (PI) controllers are used to update applied voltage to the stator windings based on the error between the reference and the measured currents. In this implementation only the i_q current reference is changed to control the torque, while the i_d current reference is always set to zero to maximize the torque per ampere.

After getting new voltage references from the PI controllers, they need to be transformed back to three phase voltages using inverse Park and inverse Clarke transformations, which can be then applied to the stator windings.

However applying the voltages directly from the inverse Park-Clarke transformations will achieve only aproximatelly 84 of the maximum voltage difference between the stator windings, which is not optimal. To maximize the voltage difference between the stator windings, Space Vector Modulation (SVPWM) can be applied to the voltage references.

Now that we can control the torque of the motor by controlling the i_q current reference, a velocity control loop can be implemented on top of the torque control by using a PI controller to update the i_q current reference based on the velocity error.

Aditionally, a position control loop can be implemented on top of the velocity control loop by using a PID controller to update the velocity reference based on the position error.

add diagram of the control loops

2.2 Clarke transformation

Clarke transformation is used to transform three phase currents (i_a, i_b, i_c) into two orthogonal currents (i_α, i_β) in a stationary reference frame, where i_α is torque generating current and i_β is flux producing current.

Clarke transformation can be represented as a matrix multiplication of the three phase currents with the following transformation matrix:

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \tag{2.1}$$

The scaling factor $\frac{2}{3}$ ensures amplitude invariance between the three-phase and i_α i_β representations.

Inverse Clarke transformation in context of FOC is used to transform two orthogonal voltages (v_α, v_β) in a stationary reference frame back to three phase voltages (v_a, v_b, v_c) that can be applied to the stator windings.

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix} \tag{2.2}$$

2.3 Park transformation

Park transformation is used to transform two orthogonal currents (i_α, i_β) in a stationary reference frame from the stator point of view into two DC currents (i_d, i_q) in a rotating reference frame from a rotor point of view, where i_d is a flux current and i_q is a torque current. To perfrom Park transformation, the electrical angle θ_e must to be calculated based on the rotor position, which can be obtained from the encoder reading.

Park transformation can be represented as a matrix multiplication of the two orthogonal currents (i_α, i_β) with the following transformation matrix:

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & \sin(\theta_e) \\ \sin(-\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} \tag{2.3}$$

In context of FOC, inverse Park transformation is used to transform two orthogonal DC voltages (v_d, v_q), which are the outputs of the PI controllers, from a rotating reference frame back to two orthogonal voltages (v_α, v_β) in a stationary reference frame.

Inverse park transformation can be represented as a matrix multiplication of the two orthogonal

DC voltages (v_d, v_q) with the following transformation matrix:

$$\begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & -\sin(\theta_e) \\ \sin(\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} v_d \\ v_q \end{bmatrix} \tag{2.4}$$

2.4 PI regulator

Anti-windup

2.5 Space vector modulation

Space vector modulation is used to maximize the voltage difference between the stator windings.

2.6 Velocity and position control loops

2.7 Electrical angle calculation

3 | Firmware implementation

3.1 Used technologies and development tools

STM32CubeMX was used to simplify the initial project setup, while the HAL library was employed to facilitate peripheral configuration and management. The firmware was implemented in the C programming language. CAN communication testing was performed using a Makerbase CANable V2.0 USB interface together with a Python based client script built on the pycan library. * Also compilation should be mantioned *

Arduino IDE was used as a serial monitor for debugging purposes.

3.2 Firmware topology

The firmware is structured into reusable modules located in the `uFOC_lib` directory. Peripheral initialization and high-level application logic are implemented in `src/main.c`.

uFOC_lib contains the following modules:

- `driver` - handles DRV8316CRRGFR driver configuration and control, ADC current sensing and PWM generation.
- `encoder` - handles MT6835GT encoder SPI configuration, positional data reading, angular velocity calculation and signal filtering.
- `foc` - implements all necessary transformations, to perform FOC control as well as goniometric functions optimizations and SVPWM calculations.
- `config` - is used to store and control global configuration parameters of the firmware. This module implements an abstraction layer to simplify the access to the configuration parameters and ensure that they are updated correctly.
- `pi_controller` - implements PI and PID controllers with anti-windup and output saturation.
- `communication` - manages CAN communication, including message parsing and sending.

The current control loop, together with current sampling and encoder reading, is executed in the ADC injected conversion complete interrupt, which is triggered by the `TIM1_CC4` event at a frequency of 5 kHz. This ensures precise alignment between current measurement and the PWM cycle.

Higher-level control loops, namely velocity and position control, are executed in a separate timer interrupt (`TIM6`) at a lower frequency of 1125 Hz, reflecting the cascaded structure of the control system.

This separation of control loops follows common practice in motor control systems, where fast inner loops regulate currents and slower outer loops regulate mechanical quantities.

3.3 MCU configuration

The uFOC driver uses an STM32F3xx microcontroller as the main processing unit. The MCU is responsible for real-time motor control, peripheral coordination, and communication with external devices. The configuration of the microcontroller is designed to meet the deterministic timing and low latency requirements of FOC.

3.3.1 System clock

The system clock is derived from the internal high-speed oscillator (HSI) operating at 8 MHz. This signal is multiplied using a phase-locked loop (PLL) with a multiplication factor of 16, resulting in a system clock frequency of 64 MHz.

The AHB bus runs at the full system clock frequency (64 MHz), while the APB1 bus is prescaled by a factor of 2 to 32 MHz, and the APB2 bus operates at 64 MHz. This configuration satisfies the peripheral frequency constraints while maintaining maximum performance for time-critical components.

Particular attention is given to peripherals involved in motor control. The advanced timer TIM1 and the ADC are both clocked at 64 MHz, ensuring high timing resolution for PWM generation and precise synchronization of current sampling.

This clock distribution enables deterministic timing of control loops and supports the strict real-time requirements of the control algorithm.

TIM6 is clocked from the APB1 timer clock. Although the APB1 peripheral clock is prescaled to 32 MHz, the timer clock is doubled to 64 MHz due to the APB1 prescaler being different from one. TIM6 is configured with a prescaler of 31 and an auto-reload value of 999, resulting in an update interrupt frequency of 2 kHz.

3.3.2 Timer configuration

The advanced timer TIM1 is used for PWM generation. It is configured in center aligned mode, which reduces switching harmonics and is commonly used in motor control applications. The timer operates at a PWM frequency of 8888 Hz.

TIM1 also generates a compare event (`CC4`) that is used to trigger ADC injected conversions. This ensures that current sampling is synchronized with the PWM cycle and occurs at a well-defined

point in time.

A separate basic timer, TIM6, is used to periodically execute higher-level control loops. TIM6 operates at a frequency of 2 kHz and triggers an interrupt used for velocity and position control. This lower update rate reflects the cascaded structure of the control system, where slower outer loops are executed less frequently than the inner current control loop.

3.3.3 ADC configuration

The ADC is configured to perform injected conversions of three phase currents. The conversions are triggered by a TIM1 compare event, ensuring that current sampling occurs at a well-defined point within the PWM cycle. This synchronization minimizes measurement noise and improves control accuracy.

The ADC conversion complete interrupt is used to process the measured currents and execute the current control loop.

3.3.4 PWM-ADC synchronization

Precise synchronization between PWM generation and current measurement is achieved by using a timer-generated trigger for ADC conversions. The TIM1 compare event (`CC4`) initiates the ADC injected sequence, and the corresponding interrupt is used to execute the control algorithm.

The trigger signal is generated by a dedicated compare channel that is not used for PWM output, allowing independent adjustment of the sampling point within the PWM cycle.

This approach ensures deterministic timing and minimizes control latency.

3.3.5 Communication interfaces

The MCU communicates with external systems primarily via the CAN interface, which is used for command exchange and system control. The CAN peripheral is configured in normal mode with automatic retransmission enabled.

The CAN peripheral is clocked from the APB1 bus operating at 32 MHz. The bit timing parameters are configured to achieve a communication speed of 500 kbit/s. This interface is used for command exchange and system control and operates independently of the real-time control loops.

A UART interface is also available and is used only for debugging and diagnostic output during development.

3.3.6 SPI interfaces

SPI communication is used to interface with external peripherals, namely the magnetic encoder and the gate driver. Both devices share a single SPI peripheral, with separate chip-select signals

used to access each device.

The SPI operates in master mode, and its configuration (clock polarity and phase) is adjusted as required by the connected peripherals.

The magnetic encoder is accessed within the real-time control loop to obtain rotor position information. The encoder update is performed inside the ADC interrupt service routine, ensuring that position data is synchronized with current measurements and control execution. As the SPI communication is performed in a blocking manner, its latency directly affects the execution time of the control loop.

In contrast, communication with the gate driver is primarily used for configuration and status monitoring. These operations are not time-critical and are performed outside the real-time control loop only during firmware initialization.

The SPI peripheral operates at a clock frequency of approximately 32 MHz, configured using a prescaler of 2. This high communication speed minimizes the duration of blocking SPI transactions, which is particularly important as encoder data acquisition is performed within the interrupt service routine.

At this speed, a the encoder read transaction takes only a few microseconds, ensuring that the communication overhead remains negligible compared to the control loop period.

3.4 Driver

3.4.1 Current sensing offset calibration

3.5 Encoder

3.5.1 Data reading

3.5.2 Direction inversion

3.5.3 CRC control

3.5.4 Velocity calculation

3.5.5 Signal filtering

3.5.6 Electrical offset calibration

3.5.7 Implemented functionality

3.5.8 Electrical offset calibration

3.6 FOC implementation

3.6.1 Current control loop

3.6.2 Velocity and position control loops

3.7 Configuration abstraction layer

4 | Conclusion

In this chapter, I conclude the thesis by providing an overview of achieved results and proposing possible future improvements of the developed method for collision avoidance described in the thesis. The main goal of this thesis was to design an algorithm which would possibly run onboard a tram and give warning of imminent rear-end collision with a forward tram. I achieved this goal by developing an algorithm which utilizes V2V communication and estimation of tram's state of motion using Kalman filter. Main contributions of this thesis are none :(.

4.1 Future work

Development of a collision warning system for trams continues within the collaboration with project partners. Since the proposed algorithm gave promising results on simulations, the main goal now is deployment and testing the collision warning system in practice. There are, of course, several problems imposed by a real environment which I neglected in the design and which need to be addressed.

Appendices

A | Supplementary figures

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Aliquam erat volutpat. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Vivamus porttitor turpis ac leo. Aliquam ornare wisi eu metus. Aliquam ante. Nulla quis diam. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Duis ante orci, molestie vitae vehicula venenatis, tincidunt ac pede. Curabitur bibendum justo non orci. Duis sapien nunc, commodo et, interdum suscipit, sollicitudin et, dolor. Phasellus et lorem id felis nonummy placerat.

Donec vitae arcu. Vivamus luctus egestas leo. Aliquam erat volutpat. Mauris dictum facilisis augue. Integer vulputate sem a nibh rutrum consequat. Proin mattis lacinia justo. Aliquam id dolor. In enim a arcu imperdiet malesuada. Duis sapien nunc, commodo et, interdum suscipit, sollicitudin et, dolor. Nunc dapibus tortor vel mi dapibus sollicitudin. Duis pulvinar. Praesent vitae arcu tempor neque lacinia pretium. Nam libero tempore, cum soluta nobis est eligendi optio cumque nihil impedit quo minus id quod maxime placeat facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Nulla quis diam.

B | Advance things

Aliquam erat volutpat. Duis risus. Fusce suscipit libero eget elit. Suspendisse sagittis ultrices augue. Nullam feugiat, turpis at pulvinar vulputate, erat libero tristique tellus, nec bibendum odio risus sit amet ante. In convallis. Fusce wisi. Nulla quis diam. Sed elit dui, pellentesque a, faucibus vel, interdum nec, diam. Pellentesque arcu. Aliquam ornare wisi eu metus. Nullam sit amet magna in magna gravida vehicula. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet ut et voluptates repudiandae sint et molestiae non recusandae. Mauris suscipit, ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis velit mauris vel metus.

References